FixFlow Web Application
(Technical Report)

Abstract
The database powered web app called FixFlow to effectively track and manage service requests has been designed in this project. With all the technological integration happening in day to day operation, there is a growing need for centralized systems to manage workflows and ensure data consistency. In this project, we built a full stack web application where a user can submit service requests and track their status, while administrators can view and update requests. We integrated a relational database with a web application to enable users to perform CRUD operations on data.

Introduction
Manual or distributed request management often results in inefficiencies, latency and inconsistency of information handling. Organizations need a defined system allowing users to submit requests and monitor the progression within a single location. FixFlow attempts to solve this problem by providing a web-based solution that combines frontend user input with backend database functionality to both manage the application and allow users to monitor and make requests. This project aimed to create a database driven application that incorporates basic concepts of database relation design, CRUD and full stack design.

Application Overview
FixFlow is a web based service request management system that allows users to make requests and communicate to a centralized database via the use of a user-friendly interface.

It currently supports:
- Creation and deletion of users (clients)
- Making requests (tickets) for services
- Displaying and monitoring the status of tickets
- Full create, read, update and delete (CRUD) functionality
- Adding comments, with timestamps, to tickets

The system saves the records in a database. Such records consist of a Request ID, a description and a status as well as timestamps. The system makes sure that all actions performed will have real-time feedback to the interface.

Overview of Technologies Utilized
We implemented the FixFlow application with a combination of Frontend, Backend and database to ensure an efficient and interactive application.

Frontend Technologies
We implemented the frontend with the help of HTML, CSS and JavaScript. The HTML was used to organize the structure of the web page. The CSS was applied for the overall look of the web page and also for the design. The JavaScript was used to execute actions when a user interacts with the web page and for server interaction between the backend.

Backend Technologies
The application logic, routing and database interaction was implemented with the help of server side framework to handle each incoming request, validating the user input and performing tasks like inserting, retrieving, deleting and updating from the database. AWS EC2 instances were used in order to host the application. One instance for the Flask server, and another holding the SQL server.

Database Management System
We used the relational database management system that supports relational models like MySQL. This system consists of various tables representing users, service requests, status records etc., and we have implemented the relations between these tables using primary and foreign keys, thus maintaining referential integrity.

Development Tools
During development, we have used Windows Subsystem for Linux (WSL) paired with its built-in nano editor to build and store the code. We have also used a local server to run and test our application.

Deployment and Hosting
FixFlow application was deployed using Amazon Web Services (AWS) by provisioning virtual server instances which were configured during in class lab sessions. The backend and the database connection were placed inside these virtual machines. We stored our application's source and configuration files on our AWS server and thus it was just like a deployment on a real server so that we were able to use it somewhere not on our local host.

Step by step Tutorial

Step 1: Access the webpage with the provided link: http://3.214.242.70/

Step 2: You are at the homepage. Click on "Departments" at the top right. Verify that your client's department exists. If it exists, **skip to step 4.**

Step 3: Click on "Create Department" and add your clients departments name (i.e. Business). Once added, go back to the home page by clicking on "Home"

Step 4: Click on "View Users" OR "Users" at the top right. Verify that your client exists. If they exist, **skip to step 6.**

Step 5: Click on "Create User". Add their full name, their email address, and select their department from the drop down. If you had just created a department, select the one you created. Verify that they were added at the bottom of the Users page.

Step 6: Go back to the home page by clicking on "Home". Click on "View Tickets" OR "Tickets" on the top right to view all created tickets. Verify that no previous ticket was created for your client regarding the current issue. If you know for a fact that this is their first ticket, feel free to skip to "Create Ticket" without viewing the tickets page.

Step 7: Click on the blue "Create Ticket" button. Fill in the information. Please find the client (user) from the dropdown. Select the department relevant to the issue. Add a title for the ticket. Provide a brief description of the issue. Select priority from: Low, Medium or High. Submit by hitting the "Create Ticket" button at the bottom of the form.

Step 8: Congratulations! You have successfully created your ticket. Now, click on "Details". As you can see, you see many details. 2 are very important. Assigned Technician and Comments. If you do not know the technician who will be assigned to this ticket, **skip to step 11**. If you do not have any comments to make at this time, **skip to step 11.**

Step 9: To assign a technician, simply click on “Edit Ticket”, click on the drop down underneath “Assign Technician” and select them. Then click on “Update Ticket”. The assigned technician is now attached to the ticket, and you will be redirected to the tickets page.

Step 10: To add any comment, click back on “Details”. Scroll down till you see “Add Comment”. Fill out your name, and comment in the boxes provided. Then submit by clicking the blue “Add Comment”. Your comment will now be visible to anyone accessing the site, and your name, as well as timestamp will be shown.

Step 11: If for whatever reason you must make a change to the ticket, go to your ticket and hit “Edit”. Now you can change the title, description, status (open, in progress, closed), priority, department, or technician. After the changes are made, hit the blue “Update Ticket”

DELETION:
Deletion is self explanatory. You are able to delete tickets, users and departments if need be. Simply go to the desired page (Tickets, Users, or Departments) by navigating using the navbar at the top right. Once you’re on the page, click on the red “Delete” button to remove the entity you’d like to delete. **WARNING**: If you delete a user with an active ticket, the ticket will also be deleted.

UPDATING:
You are able to update tickets quite easily. Simply navigate to the tickets page, and hit the blue “Edit” button on the desired ticket. This is useful when you need to assign a technician to a ticket, or change the status of the ticket from open to in progress, or even closed for example.

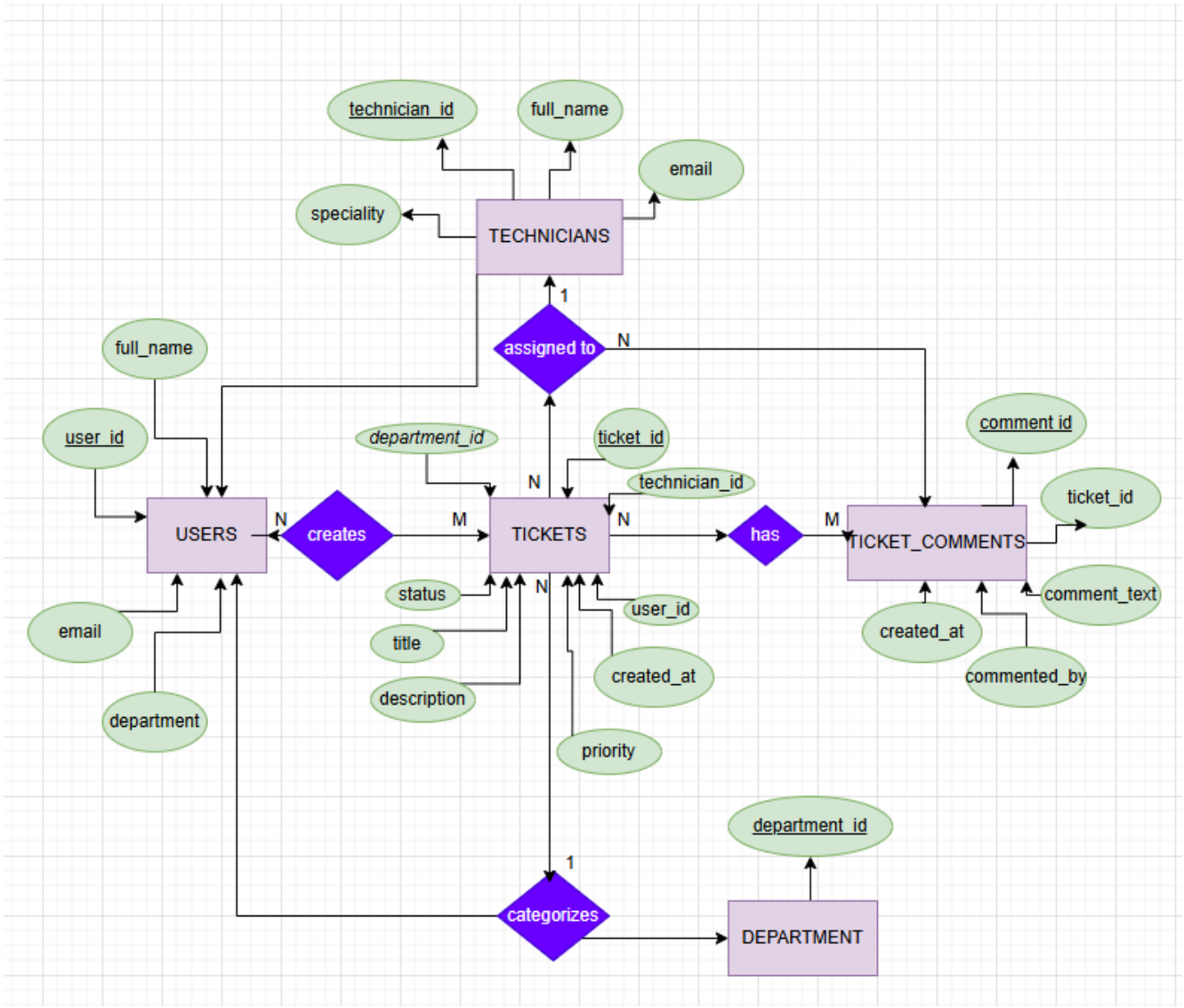
COMMENTS:
Comments are able to be added to a ticket as these comments can give clarity to an issue. Simply go to the tickets page, and hit “Details” on the desired ticket. Scroll to the Add Comments section and add your comments. They will automatically be added in real-time, and you can also view previous comments in the same section they are added.

Conclusion
We’ve explored the power of database driven systems with this FixFlow web application, which uses the ability to organize and display data in a real time fashion to manage service requests. We created a structured data system that integrates frontend activity with a backend database. We’ve shown how to build a well-organized workflow system that is scalable by working both ends of a modern web application: the frontend and the full stack.

Appendix

ER Diagram

The diagram shows the relationship we have established between the main entities: User,Request and Status. Each user could potentially have numerous requests, and each request has one status.



Relation Schema:
USERS(user_id PK, full_name, email, department)

TECHNICIANS(technician_id PK, full_name, email, specialty)

DEPARTMENTS(department_id PK, name)

TICKETS(ticket_id PK, user_id FK, technician_id FK, department_id FK, title, description, status, priority, created_at)
TICKETS.user_id → USERS.user_id
TICKETS.technician_id → TECHNICIANS.technician_id
TICKETS.department_id → DEPARTMENTS.department_id

TICKET_COMMENTS(comment_id PK, ticket_id FK, comment_text, commented_by, created_at)
TICKET_COMMENTS.ticket_id → TICKETS.ticket_id

SQL Queries

Q.1 – Retrieve all tickets with requester, technician information and department information

```
SELECT t.ticket_id, t.title, t.status, t.priority, u.full_name AS user_name, tech.full_name AS technician_name, d.name AS department_name
FROM tickets t
JOIN users u ON t.user_id = u.user_id
LEFT JOIN technicians tech ON t.technician_id = tech.technician_id
LEFT JOIN departments d ON t.department_id = d.department_id
ORDER BY t.ticket_id;
```

*Used on ticket page to display all tickets, along with the user who created, the tech assigned to it, and department

Q.2 – Retrieve all users

```
SELECT * FROM users
ORDER BY full_name;
```

*Used on the Users page and when using the dropdown in Create Ticket form

Q.3 – Insert a new user

```
INSERT INTO users (full_name, email, department)
VALUES (%s, %s, %s);
```

*Used when creating a new user through the Create User form

Q.4 – Retrieve all technicians

```
SELECT * FROM technicians
ORDER BY full_name;
```

*Used when populating the technician dropdown in Edit Ticket form

Q.5 – Insert a new ticket

```
INSERT INTO tickets (user_id, department_id, title, description, priority, status)
VALUES (%s, %s, %s, %s, 'Open');
```

*Used when creating a new ticket

Q.6 – Retrieve a specific ticket with user and technicians' details

```
SELECT t.ticket_id, t.user_id, t.technician_id, t.title, t.description, t.status, t.priority, t.created_at,
u.full_name AS user_name, u.email AS user_email, u.department, tech.full_name AS
technician_name, d.name as department_name
FROM tickets t
JOIN users u ON t.user_id = u.user_id
LEFT JOIN technicians tech ON t.technician_id = tech.technician_id
LEFT JOIN departments d on t.department_id = d.department_id
WHERE t.ticket_id = %s;
```

*Used on Ticket Details page to display full information for one ticket, including requester and technician information

Q.7 – Update a ticket

```
UPDATE tickets
SET title = %s, description = %s, status = %s, priority = %s, technician_id = %s, department_id
= %s
WHERE ticket_id = %s;
```

*Used when editing a ticket

Q.8 – Delete a ticket

```
DELETE FROM tickets
WHERE ticket_id = %s;
```

*Used when deleting a ticket from Tickets page

Q.9 – Retrieve comments for a ticket
SELECT * FROM ticket_comments
WHERE ticket_id = %s
ORDER BY created_at DESC, comment_id DESC;

*Used on Ticket Details page to display all comments for a ticket

Q.10 – Insert a comment
INSERT INTO ticket_comments (ticket_id, comment_text, commented_by)
VALUES (%s, %s, %s);

*Used when adding a comment to a ticket

Q.11 – Delete a comment
DELETE FROM ticket_comments
WHERE comment_id = %s;

*Used when deleting a comment from Ticket Details page

Q.12 – Delete a user
DELETE FROM users
WHERE user_id = %s;
*Used when deleting a user from the Users page
Q.13 - Retrieve all departments
SELECT * FROM departments
ORDER BY name;

*Used on departments page to show all departments

Q.14 – Insert a new department
INSERT INTO departments (name)
VALUES (%s);

*Used on create department page to create department

Q15 – Delete a department
DELETE FROM departments
WHERE department_id = %s;

*Used when deleting a department